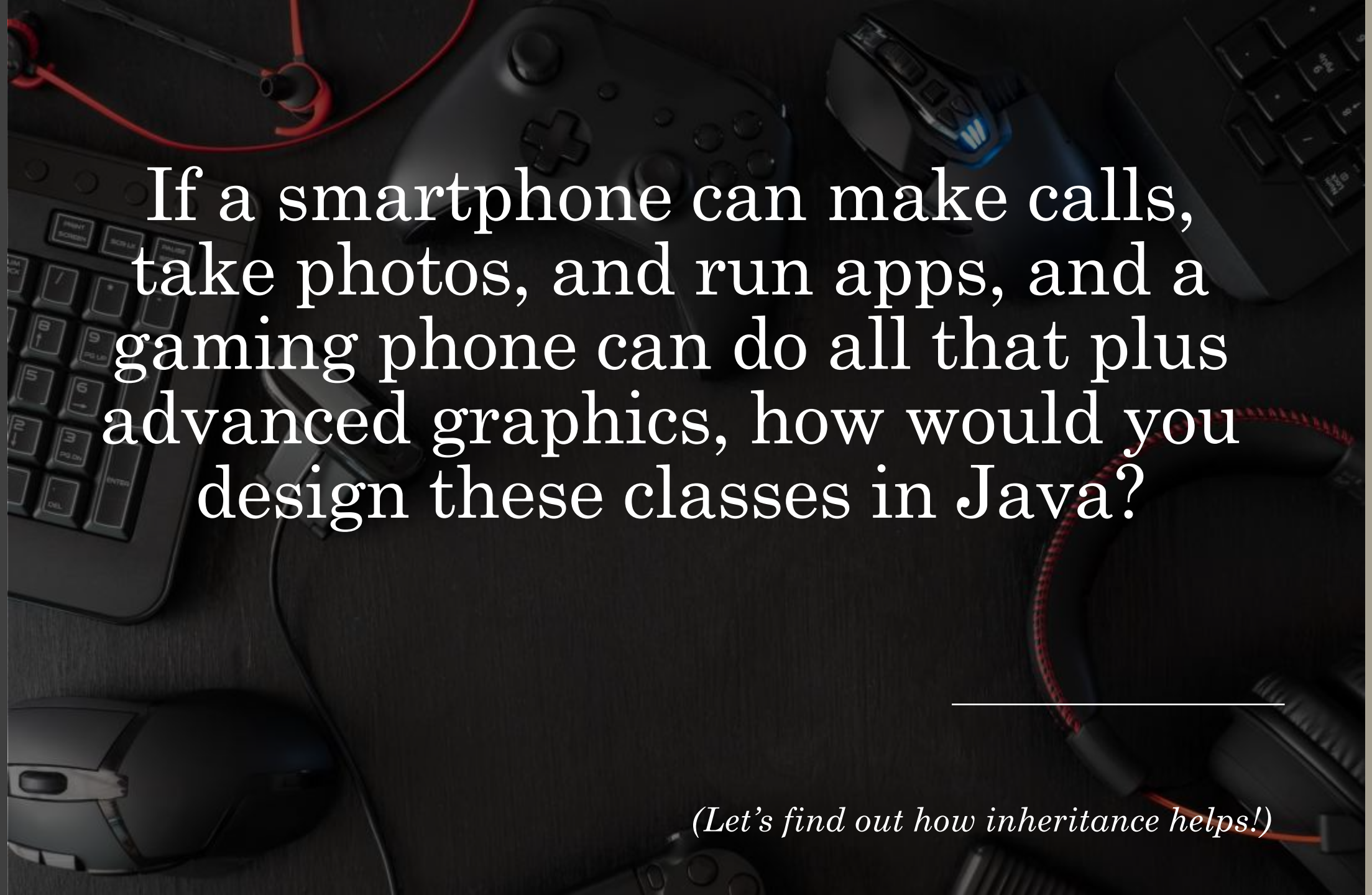


If a smartphone can make calls, take photos, and run apps, and a gaming phone can do all that plus advanced graphics, how would you design these classes in Java?

(Let's find out how inheritance helps!)

What is Inheritance?

- Inheritance allows a **class to acquire properties and behaviors of another class**.
- **Parent class (superclass):** The original class.
- **Child class (subclass):** The class that inherits.

Java Syntax

```
class Parent { }
class Child extends
Parent { }
```

Extends keyword

- Used to provide inheritance in java
- A extends B; means A is a subclass(specialized version) of B and B is super class
- Extends functionality

Real-Life Example

Vehicle → speed, fuel, start(), stop()

Car → inherits Vehicle, adds AC, music system

Bike → inherits Vehicle, adds helmetCompartment

Code Sample

```
class Vehicle {
    void start() { System.out.println("Vehicle
starting"); }
}
class Car extends Vehicle {
    void playMusic() { System.out.println("Playing
music"); }
}
```

Example 1: School → Student (Accessing Methods)

```
// Parent class
class School {
    void schoolName() {
        System.out.println("Welcome to Greenfield School");
    }
}

// Child class
class Student extends School {
    void studentName() {
        System.out.println("Student: Ayesha");
    }
}

// Main class
public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.schoolName();   // inherited
        s.studentName();  // own method
    }
}
```

Example 2: Vehicle → Car (Accessing Variables)

```
// Parent class
class Vehicle {
    String car_name = "Honda";
}

// Child class
class Car extends Vehicle {
    String brakes = "ABS";
}

// Main class
public class Main {
    public static void main(String[] args) {
        Car c = new Car();
        System.out.println(c.car_name);      // inherited
        System.out.println(c.brakes);   // own
    }
}
```

Example 3: Accessing both methods and variables

```
// Parent class
class Animal {
    String type = "Mammal";   // variable
    void eat() {
        System.out.println("Animal is eating");
    }
}

// Child class
class Dog extends Animal {
    String breed = "Bulldog";  // own variable
    void bark() {
        System.out.println("Dog is barking");
    }
}

// Main class
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        System.out.println("Type: " + d.type);   // inherited variable
        System.out.println("Breed: " + d.breed); // own variable
        d.eat();  // inherited method
        d.bark(); // own method
    }
}
```

Example 4

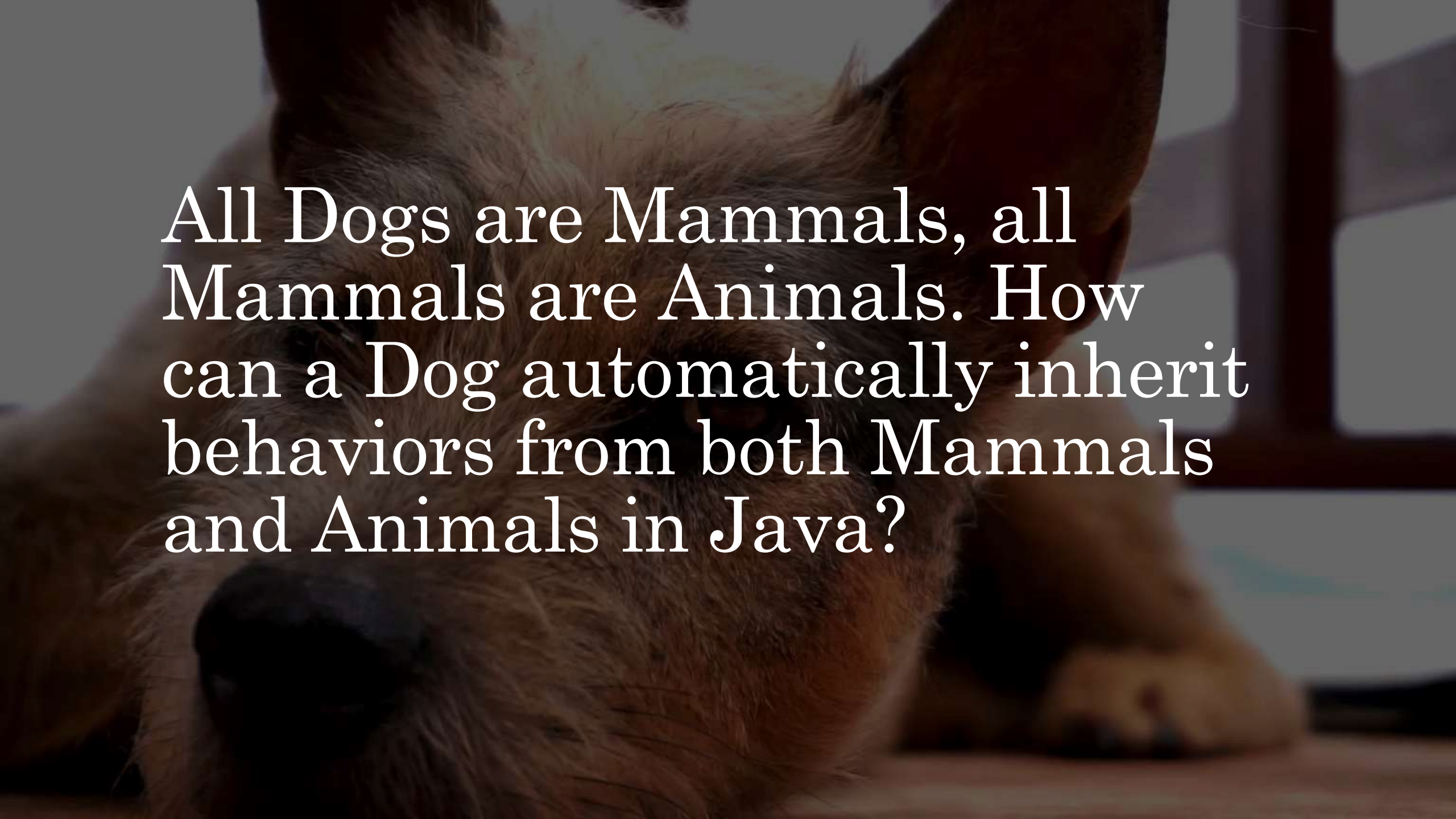
```
class Smartphone {
    void call() {
        System.out.println("Call");
    }

    void camera() {
        System.out.println("Take Photo");
    }

    void msg() {
        System.out.println("Send Message");
    }
}

class Gaming extends Smartphone {
    void graphics() {
        System.out.println("Play Games");
    }
}

public class Main {
    public static void main(String[] args) {
        Gaming obj = new Gaming();
        obj.graphics();
        obj.camera();
    }
}
```

Key Benefits

Code Reusability: No need to rewrite shared features.

Logical Hierarchy: Models real-world relationships.

Extensibility: Easily add new features in child classes.


All Dogs are Mammals, all Mammals are Animals. How can a Dog automatically inherit behaviors from both Mammals and Animals in Java?

Types of Inheritance

Single Inheritance – One child inherits from **one parent**

- *Example:* Car → Vehicle

Multilevel Inheritance – Chain: **Grandparent → Parent → Child**

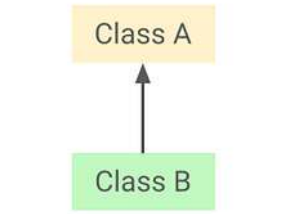
- *Example:* Dog → Mammal → Animal

Hierarchical Inheritance – One parent → multiple children

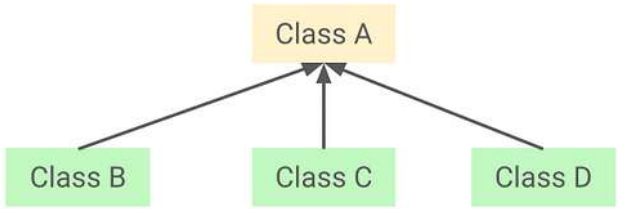
- *Example:* Vehicle → Car, Bike

Hybrid Inheritance (Combination) – Combination of above types

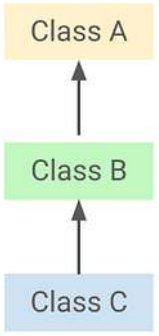
Types of Inheritance in OOP



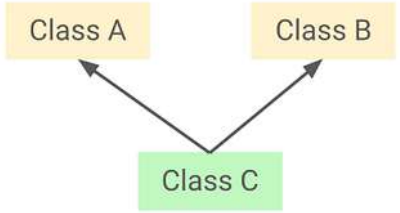
Single Inheritance



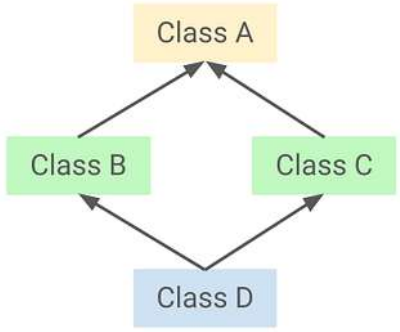
Hierarchical inheritance



Multilevel Inheritance



Multiple Inheritance



Hybrid Inheritance

Multiple Inheritance in Java

In java multiple inheritance is not supported

Whereas Multilevel Inheritance is supported

Java uses interfaces to solve this isuue

Inheritance Support in Java

Inheritance Type	Supported with Classes?	Supported via Interfaces?
Single	Yes	Yes
Multilevel	Yes	Yes
Hierarchical	Yes	Yes
Multiple	No	Yes
Hybrid	No	Yes

Dog, Mammal and Animal Problem

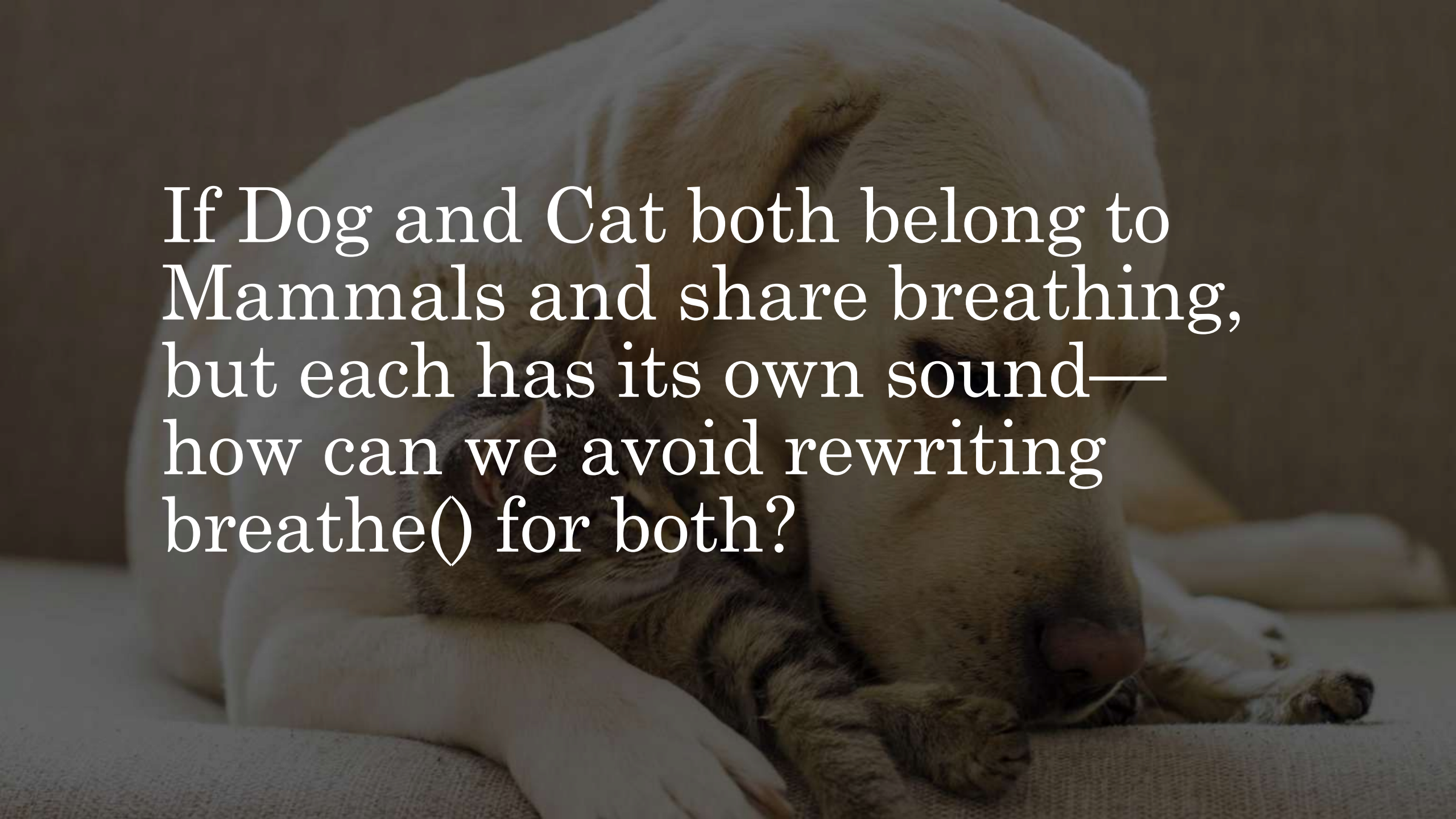
- We would use multi-level inheritance.
- Multilevel inheritance occurs when a class is derived from another derived class
- Creating a "chain" of inheritance
- This allows a subclass to inherit properties from its immediate parent and all ancestors above it.
- **Grandparent Class:** Animal (Base Class)
- **Parent Class:** Mammal (Derived from Animal)
- **Child Class:** Dog (Derived from Mammal)

Code Solution

```
// Top-level Superclass
class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}

// Intermediate Subclass
class Mammal extends Animal {
    void breathe() {
        System.out.println("Breathes air...");
    }
}

// Bottom-level Subclass
class Dog extends Mammal {
    void bark() {
        System.out.println("Barking...");
    }
}

// Main class
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();      // from Animal
        d.breathe();  // from Mammal
        d.bark();     // from Dog
    }
}
```

Key Takeaways

- **Transitive Nature:** If Dog extends Mammal and Mammal extends Animal, then Dog is also an Animal.
- **Code Reusability:** Dog automatically gains access to eat() and breathe() without redefining them.
- **Single Parent Rule:** Even in a long chain, each class has exactly **one** immediate superclass, satisfying Java's restriction against multiple class inheritance.
- **Memory Efficiency:** Only one object of the Dog class is created, but it contains the state and behavior of the entire lineage.


If Dog and Cat both belong to Mammals and share breathing, but each has its own sound—how can we avoid rewriting breathe() for both?

Hierarchical Inheritance

One parent class → **multiple child classes**

Common behavior is **shared**

Each child has its **own unique behavior**

Example

```
// Parent class
class Mammal {
    void breathe() {
        System.out.println("Breathing...");
    }
}

// Child class 1
class Dog extends Mammal {
    void bark() {
        System.out.println("Woof!");
    }
}

// Child class 2
class Cat extends Mammal {
    void meow() {
        System.out.println("Meow!");
    }
}

// Main class
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();

        // Dog actions
        d.breathe(); // inherited
        d.bark();    // own

        // Cat actions
        c.breathe(); // inherited
        c.meow();    // own
    }
}
```

Key Idea

- Dog and Cat both inherit breathing from Mammal, but keep their own behaviors (bark, meow).

Summary

Inheritance Type	Structure / Example	Key Idea / Behavior
Single	Mammal → DogDog inherits breathe() from Mammal	One parent → one child; child reuses parent behavior
Multilevel	Animal → Mammal → DogDog inherits eat() & breathe()	Chain of inheritance; child inherits from parent & grandparent
Hierarchical	Mammal → Dog, Mammal → CatBoth inherit breathe()	One parent → multiple children; each child adds own behavior

What will be output of the following?

```
class Animal {
    String type = "Mammal";
}

class Dog extends Animal {
    String type = "Bulldog";

    void showType() {
        System.out.println(type);
}
}
public class Main {
    public static void main(String[]
args) {
        Dog d = new Dog();
        d.showType();
    }
}
```

What will be output of the following?

```
class Animal {
    void sound(){
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    void sound(){
        System.out.println("Dog barks");
    }
    void makeSound() {
        sound();
}
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.makeSound();
    }
}
```

Variable Hiding vs Method Overriding

Variable Hiding : If a child class has a variable with the same name as the parent class variable, the child variable hides the parent variable.

Method Overriding : If a child class has a method with the same name and parameters as the parent class method, the child method replaces the parent method.

Variable Hiding vs Method Overriding

Concept	Variables	Methods
Same name in parent and child	Allowed	Allowed
What happens?	Child variable hides parent variable	Child method overrides parent method
Official term	Variable Hiding	Method Overriding

Overloading vs Overriding

Method Overriding: When a child class provides its own implementation of a method that is already defined in the parent class with the **same name, return type, and parameters**.

Method Overloading: When a class has **multiple methods with the same name but different parameters** (type, number, or order) within the **same class**.

What if both the parent and child class have a variable with the same name? How can the child still access the parent's version?

Super Keyword

What is super Keyword?

super refers to the **parent class object**

It is used to:

- Access parent class variables
- Access parent class methods
- Call parent class constructor

Example with Variable

```
class Animal {
    String type = "Mammal";
}

class Dog extends Animal {
    String type = "Bulldog";

    void showType() {
        System.out.println(type);        // child variable
        System.out.println(super.type);  // parent
variable
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.showType();
    }
}
```

Example with method

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }

    void display() {
        sound();        // child method
        super.sound();  // parent method
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.display();
    }
}
```

Example with Constructor

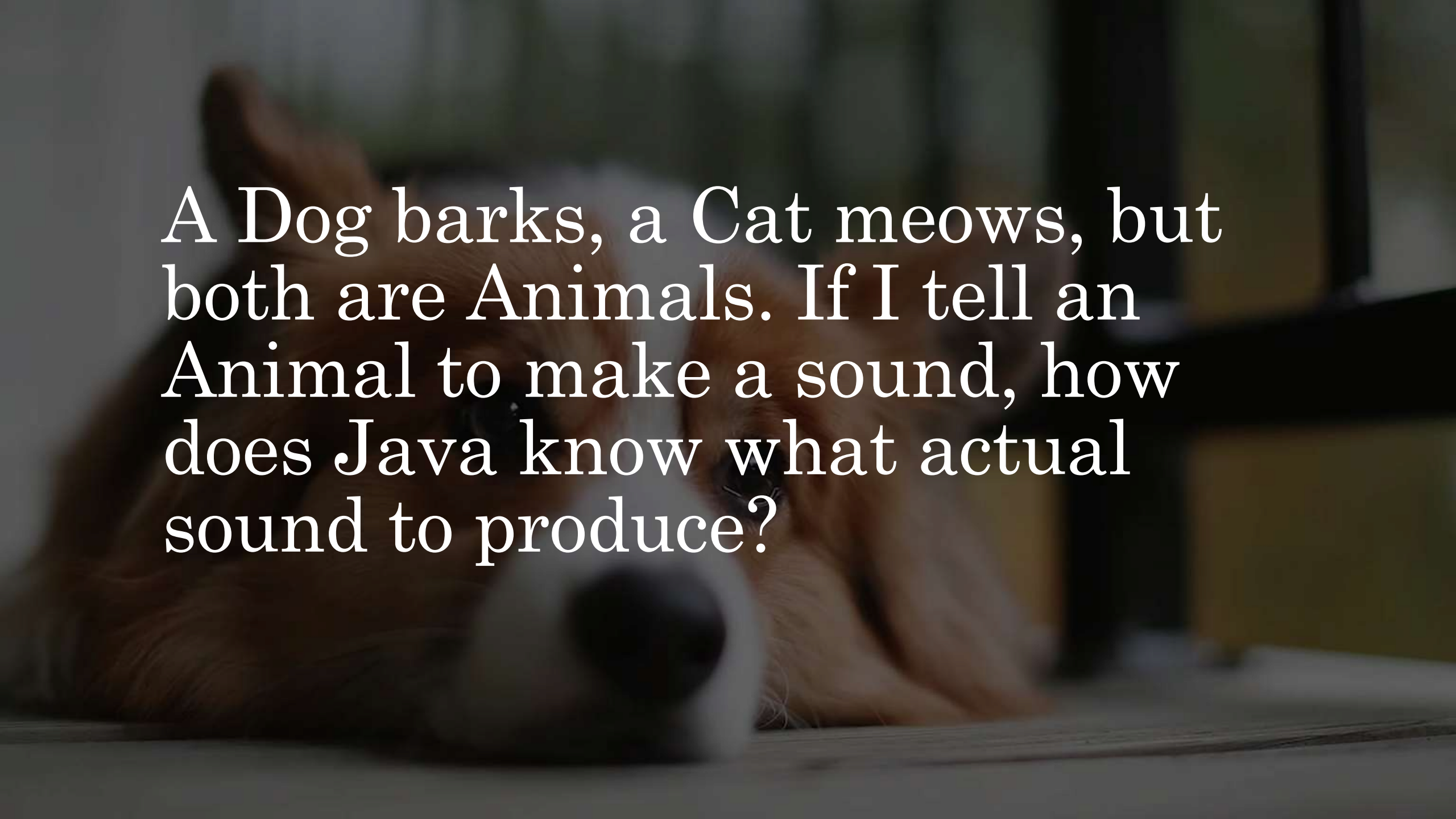
```
class Animal {
    Animal() {
        System.out.println("Animal constructor called ");
    }
}

class Dog extends Animal {
    Dog() {
        super();
        System.out.println("Dog constructor called");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
    }
}
```

Example with Constructor

super() becomes useful when the parent class has:

1. A parameterized constructor
2. Multiple constructors
3. Important initialization logic that you want to control explicitly

```
class Animal {
    Animal(String type) {
        System.out.println("Animal type: " + type);
    }
}

class Dog extends Animal {
    Dog() {
        super("Mammal");
        System.out.println("Dog constructor called");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
    }
}
```


A Dog barks, a Cat meows, but both are Animals. If I tell an Animal to make a sound, how does Java know what actual sound to produce?

What is Polymorphism?

- Polymorphism means **“many forms”**.
- In Java, it allows **one reference type to take multiple forms**.
- Two main types:
  1. **Compile-time (Method Overloading)** – same method name, different parameters
  2. **Run-time (Method Overriding)** – child class decides which method runs

Real-life Intuition

Animal a = new Dog(); →
a.sound() → **Dog barks**

Animal a = new Cat(); →
a.sound() → **Cat meows**

Same Animal reference, different behaviors → **polymorphism**

Example (Run-time Polymorphism)

Key Idea: One reference type (Animal) → multiple forms (Dog, Cat)

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    void sound() {
        System.out.println("Cat meows");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a1 = new Dog();
        Animal a2 = new Cat();

        a1.sound(); // Dog barks
        a2.sound(); // Cat meows
    }
}
```

Compile-time Polymorphism (Method Overloading)

- Same method name add() → different forms depending on **parameters**
- Determined **at compile time**, unlike run-time polymorphism (overriding)
- Same method name but different parameters (number or type) in the same class.
- Resolved at compile time, hence "compile-time polymorphism."

```
class Calculator {

    // Addition of two integers
    int add(int a, int b) {
        return a + b;
    }

    // Addition of three integers
    int add(int a, int b, int c) {
        return a + b + c;
    }

    // Addition of two doubles
    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        System.out.println(calc.add(5, 10));       // Calls add(int, int)
        System.out.println(calc.add(2, 4, 6));     // Calls add(int, int, int)
        System.out.println(calc.add(3.5, 2.5));    // Calls add(double, double)
    }
}
```

What if we want a method or class to stay exactly as it is, and prevent anyone from changing it?

Final Keyword

What is final?

Final Variable: Cannot change the value

Final Method: Cannot be **overridden** by a child class

Final Class: Cannot be **extended** by any other class

Example: Final Method

Key point: Dog **cannot override** the breathe() method.

```
// Parent class
class Animal {
    // This method cannot be overridden
    final void breathe() {
        System.out.println("Breathing...");
    }
}

// Child class
class Dog extends Animal {
    // Uncommenting the below method will cause a compilation error
    /*
    void breathe() {
        System.out.println("Dog breathing...");
    }
    */
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.breathe();  // Calls parent class method
    }
}
```

Final Class Example

- Key point: Animal **cannot be subclassed**, so no class can extend it.

```
// Final class cannot be extended
final class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}

// Child class attempting to extend Animal will cause an error
/*
class Dog extends Animal {
    void bark() {
        System.out.println("Woof!");
    }
}
*/

public class Main {
    public static void main(String[] args) {
        Animal a = new Animal();
        a.eat();
    }
}
```

Industrial Applications

Concept	Industrial Application Example
Inheritance	**Banking Systems:** Base Account class → SavingsAccount, CurrentAccount share deposit/withdraw logic
Method Overriding	**E-commerce Platforms:** Payment class → overridden in CreditCardPayment, PayPalPayment for different payment behaviors
Variable Hiding	**IoT Device Configurations:** Child device hides parent default settings but can access via super
super Keyword	**Robotics Software:** Robot subclasses override sensor methods but call super.readSensor() for base calibration

Industrial Applications

Concept	Industrial Application Example
Polymorphism	**Autonomous Vehicles:** Vehicle v = new Car(); v.drive(); → same interface, multiple forms (Car, Truck, Drone)
Compile-time Polymorphism (Overloading)	**Math/Finance Libraries:** calculate() method overloaded for interest, tax, or loan computations
Final Method	**Security Modules:** authenticate() locked in parent class to prevent tampering in subclasses
Final Class	**Core Utility Libraries:** Logger or Math class cannot be extended, ensuring consistency and thread-safety